Square Root 예제 분석 (Square Root Example Analysis)

개요

이 문서에서는 Plonky2의 `square_root.rs` 예제를 상세히 분석합니다. 이 예제는 "나는 이 필드 원소의 제곱근을 알고 있다"는 명제를 증명하는 방법을 보여줍니다.

이 예제의 핵심은 **Custom Generator**의 사용입니다. Generator는 회로 외부에서 복잡한 계산을 수행하고 그 결과를 Witness로 제공하는 메커니즘입니다. 제곱근 계산은 회로 내에서 직접 구현하기 어렵지만, Generator를 사용하면 효율적으로 처리할 수 있습니다.

이 예제를 통해 다음 개념들을 학습할 수 있습니다:

- Custom Generator의 개념과 필요성
- SimpleGenerator trait 구현 방법
- Generator의 Serialization/Deserialization
- Generator를 사용한 Witness 생성 과정
- 회로 내 검증과 회로 외 계산의 분리

Custom Generator란?

Generator의 역할

Generator는 **Witness 생성 과정에서 자동으로 실행되는 코드**입니다. 다음과 같은 상황에서 필요합니다:

1. **복잡한 계산**: 제곱근, 나눗셈, 모듈러 역원 등 회로로 표현하기 어려운 연산
2. **비결정적 계산**: 여러 가능한 답 중 하나를 선택해야 하는 경우

3. **효율성**: 회로 크기를 줄이기 위해 일부 계산을 외부에서 수행

Generator vs 회로 제약조건

중요한 점은 Generator는 **계산만 수행**하고, **검증은 회로가 수행**한다는 것입니다:

- **Generator**: `x_squared` 가 주어졌을 때 `x = sqrt(x_squared)` 계산
- **회로**: `x * x = x_squared` 검증

이렇게 하면:

- 제곱근 계산의 복잡성은 회로 밖에서 처리
- 회로는 단순한 곱셈 검증만 수행 (훨씬 효율적)

전체 코드 구조

```rust
use core::marker::PhantomData;

use anyhow::Result;

use plonky2::field::types::{PrimeField, Sample};

use plonky2::hash::hash_types::RichField;

use plonky2::iop::generator::{GeneratedValues, SimpleGenerator};

use plonky2::iop::target::Target;

use plonky2::iop::witness::{PartialWitness, PartitionWitness, WitnessWrite};

use plonky2::plonk::circuit_builder::CircuitBuilder;

use plonky2::plonk::circuit_data::{CircuitConfig, CommonCircuitData};

use plonky2::plonk::config::{GenericConfig, PoseidonGoldilocksConfig};

use plonky2::util::serialization::{Buffer, IoResult, Read, Write};

use plonky2_field::extension::Extendable;
```

```rust
// Custom Generator 정의
#[derive(Debug, Default)]
struct SquareRootGenerator<F: RichField + Extendable<D>, const D: usize> {
x: Target,
x_squared: Target,
_phantom: PhantomData<F>,
}

// SimpleGenerator trait 구현
impl<F: RichField + Extendable<D>, const D: usize> SimpleGenerator<F, D>
for SquareRootGenerator<F, D>
{
fn  id(&self) -> String { ... }
fn  dependencies(&self) -> Vec<Target> { ... }
fn  run_once(&self, witness: &PartitionWitness<F>, out_buffer: &mut GeneratedVal
fn  serialize(&self, dst: &mut Vec<u8>, _common_data: &CommonCircuitData<F, D>)
fn  deserialize(src: &mut Buffer, _common_data: &CommonCircuitData<F, D>) -> IoR
}

// Custom Serializer 정의
#[derive(Default)]
pub  struct CustomGeneratorSerializer<C: GenericConfig<D>, const D: usize> {
```

```
pub _phantom: PhantomData<C>,
}

fn main() -> Result<()> {
// 1. 설정
const D: usize = 2;
type C = PoseidonGoldilocksConfig;
type F = <C as GenericConfig<D>>::F;
// 2. 회로 구성
let config = CircuitConfig::standard_recursion_config();
let mut builder = CircuitBuilder::<F, D>::new(config);
let x = builder.add_virtual_target();
let x_squared = builder.square(x);
builder.register_public_input(x_squared);
// 3. Custom Generator 추가
builder.add_simple_generator(SquareRootGenerator::<F, D> {
x,
x_squared,
_phantom: PhantomData,
});
// 4. Witness 설정 (x_squared만 설정, x는 Generator가 계산)
let x_squared_value = { /* 이차잉여 생성 */ };
let mut pw = PartialWitness::new();
```

```
pw.set_target(x_squared, x_squared_value)?;

// 5. 증명 생성 및 검증

let  data = builder.build::<C>();

let  proof = data.prove(pw.clone())?;

data.verify(proof)

}
```

SquareRootGenerator 구조체 분석

구조체 정의

```
#[derive(Debug, Default)]

struct SquareRootGenerator<F: RichField + Extendable<D>, const D: usize> {

x: Target,

x_squared: Target,

_phantom: PhantomData<F>,

}
```

필드 설명:

`x: Target`

- Generator가 **계산할** 값 (제곱근)
- 회로에서는 이미 정의된 Target이지만, 값은 아직 설정되지 않음
- Generator가 실행될 때 이 Target에 값을 설정

`x_squared: Target`

- Generator의 **입력** 값 (제곱 값)
- 사용자가 Witness에 이미 설정한 값
- Generator는 이 값을 읽어서 제곱근을 계산

`_phantom: PhantomData<F>`

- Rust의 타입 시스템을 위한 마커
- 구조체가 `F` 타입을 사용한다는 것을 컴파일러에게 알림
- 실제 메모리를 차지하지 않음 (zero-sized type)

제네릭 파라미터:

- `F: RichField + Extendable<D>` : 사용할 필드 타입 (예: Goldilocks Field)
- `const D: usize` : Extension degree (일반적으로 2)

왜 PhantomData가 필요한가?

Rust의 타입 시스템은 구조체가 사용하는 모든 제네릭 타입이 필드에 나타나야 합니다. 하지만 `SquareRootGenerator` 는 `F` 타입을 직접 저장하지 않고, 런타임에만 사용합니다. `PhantomData<F>` 를 추가하면:

1. 컴파일러에게 "이 구조체는 `F` 타입과 관련이 있다"고 알림
2. 타입 안전성 보장
3. 메모리 오버헤드 없음 (크기가 0)

SimpleGenerator Trait 구현

`SimpleGenerator` 는 Witness 생성기를 정의하는 핵심 trait입니다. 5개의 메서드를 구현해야 합니다.

1. id() 메서드

```
fn id(&self) -> String {
"SquareRootGenerator".to_string()
}
```

목적: Generator의 고유 식별자를 반환합니다.

사용처:

- 디버깅 및 로깅
- Serialization/Deserialization 시 Generator 타입 식별
- 에러 메시지에서 어떤 Generator가 문제를 일으켰는지 추적

주의사항:

- 각 Generator 타입마다 고유한 ID를 사용해야 함
- 일반적으로 구조체 이름을 그대로 사용

2. dependencies() 메서드

```
fn dependencies(&self) -> Vec<Target> {
vec![self.x_squared]
}
```

목적: 이 Generator가 실행되기 전에 값이 설정되어야 하는 Target들을 반환합니다.

의존성 그래프:

```
x_squared (사용자 설정)
↓
```

```
SquareRootGenerator 실행

↓

x (Generator가 계산)
```

중요성:

- Witness 생성 과정에서 Generator들의 실행 순서를 결정
- 의존성이 있는 Generator는 의존하는 값이 모두 설정된 후에 실행됨
- 순환 의존성이 있으면 에러 발생

예시:

- Generator A가 Target `t1` 을 계산하고 Target `t2` 에 의존
- Generator B가 Target `t2` 를 계산
- 실행 순서: B → A

3. run_once() 메서드

```rust
fn run_once(
&self,
witness: &PartitionWitness<F>,
out_buffer: &mut GeneratedValues<F>,
) -> Result<()> {
let x_squared = witness.get_target(self.x_squared);
let x = x_squared.sqrt().unwrap();

println!("Square root: {x}");
```

```
out_buffer.set_target(self.x, x)

}
```

목적: 실제 계산을 수행하는 핵심 메서드입니다.

파라미터:

`witness: &PartitionWitness<F>`

- 현재까지 설정된 Witness 값들을 포함
- `get_target(target)` 메서드로 특정 Target의 값을 읽을 수 있음
- 읽기 전용 (read-only)

`out_buffer: &mut GeneratedValues<F>`

- 이 Generator가 계산한 값들을 저장하는 버퍼
- `set_target(target, value)` 메서드로 Target에 값을 설정
- 쓰기 전용 (write-only)

실행 과정:

1. **입력 읽기**:

```
let  x_squared = witness.get_target(self.x_squared);
```

- `x_squared` Target의 값을 Witness에서 읽음
- 이 값은 사용자가 `PartialWitness` 에 미리 설정한 값

2. **계산 수행**:

```
let  x = x_squared.sqrt().unwrap();
```

- 필드 원소의 제곱근을 계산
- `sqrt()` 메서드는 `Option<F>` 를 반환 (제곱근이 없을 수 있음)
- `unwrap()` 으로 값을 추출 (이차잉여가 아니면 패닉)

3. **결과 저장**:

```
out_buffer.set_target(self.x, x)
```

- 계산된 제곱근을 `x` Target에 설정
- 이 값은 나중에 회로의 제약조건 검증에 사용됨

에러 처리:

- `Result<()>` 를 반환하므로 에러 전파 가능
- 계산이 불가능한 경우 `Err` 를 반환하여 증명 생성 실패

디버깅:

```
println!("Square root: {x}");
```

- 계산된 값을 출력하여 디버깅 용이
- 프로덕션 코드에서는 제거하거나 로깅 프레임워크 사용 권장

4. serialize() 메서드

```
fn  serialize(&self, dst: &mut Vec<u8>, _common_data: &CommonCircuitData<F, D>)

dst.write_target(self.x)?;

dst.write_target(self.x_squared)
```

```
}
```

목적: Generator를 바이트 스트림으로 직렬화합니다.

필요성:

- 회로 데이터(`CircuitData`)를 파일에 저장하거나 네트워크로 전송할 때 필요
- Generator도 회로의 일부이므로 함께 직렬화되어야 함
- 나중에 동일한 회로를 복원할 수 있음

파라미터:

`dst: &mut Vec<u8>`

- 직렬화된 데이터를 저장할 바이트 벡터
- `Write` trait의 메서드들을 사용하여 데이터 기록

`_common_data: &CommonCircuitData<F, D>`

- 회로의 공통 데이터 (이 예제에서는 사용하지 않음)
- 일부 Generator는 회로 정보를 참조하여 직렬화할 수 있음

구현:

```
dst.write_target(self.x)?;

dst.write_target(self.x_squared)
```

- `write_target()` : Target을 바이트로 변환하여 기록
- `?` 연산자로 에러 전파
- 필드 순서대로 직렬화 (역직렬화 시 동일한 순서로 읽어야 함)

Target 직렬화 형식:

- Target은 내부적으로 인덱스나 와이어 정보를 포함

- `write_target()` 은 이 정보를 효율적으로 인코딩

5. deserialize() 메서드

```
fn deserialize(src: &mut Buffer, _common_data: &CommonCircuitData<F, D>) -> IoR
let x = src.read_target()?;
let x_squared = src.read_target()?;
Ok(Self {
x,
x_squared,
_phantom: PhantomData,
})
}
```

목적: 바이트 스트림에서 Generator를 복원합니다.

파라미터:

`src: &mut Buffer`

- 직렬화된 데이터를 읽을 버퍼
- `Read` trait의 메서드들을 사용하여 데이터 읽기

반환값: `IoResult<Self>`

- 성공 시 복원된 Generator 인스턴스
- 실패 시 IO 에러

구현:

```
let x = src.read_target()?;
let x_squared = src.read_target()?;
```

- `read_target()` : 바이트에서 Target을 복원
- 직렬화와 **동일한 순서**로 읽어야 함
- `?` 연산자로 에러 전파

구조체 생성:

```
Ok(Self {
x,
x_squared,
_phantom: PhantomData,
})
```

- 읽은 데이터로 새 Generator 인스턴스 생성
- `PhantomData` 는 기본값 사용 (크기가 0이므로 직렬화 불필요)

중요 사항:

- `serialize()` 와 `deserialize()` 는 **정확히 대칭**이어야 함
- 필드 추가/제거 시 버전 관리 필요
- 호환성 깨짐을 방지하기 위해 신중하게 설계

CustomGeneratorSerializer 구현

Generator를 직렬화/역직렬화하려면 `WitnessGeneratorSerializer` 를 구현해야 합니다.

구조체 정의

```
#[derive(Default)]
pub struct CustomGeneratorSerializer<C: GenericConfig<D>, const D: usize> {
pub _phantom: PhantomData<C>,
}
```

목적: 여러 Generator 타입을 처리할 수 있는 Serializer를 정의합니다.

WitnessGeneratorSerializer 구현

```
impl<F, C, const D: usize> WitnessGeneratorSerializer<F, D> for CustomGeneratorS
where
F: RichField + Extendable<D>,
C: GenericConfig<D, F = F> + 'static,
C::Hasher: AlgebraicHasher<F>,
{
impl_generator_serializer! {
CustomGeneratorSerializer,
DummyProofGenerator<F, C, D>,
ArithmeticBaseGenerator<F, D>,
ConstantGenerator<F>,
PoseidonGenerator<F, D>,
PoseidonMdsGenerator<D>,
```

```
RandomValueGenerator,
SquareRootGenerator<F, D>
}
}
```

`impl_generator_serializer!` 매크로:

이 매크로는 다음을 자동으로 생성합니다:

1. **Generator 타입 태그**: 각 Generator에 고유한 숫자 ID 할당
2. **직렬화 로직**: Generator 타입에 따라 적절한 `serialize()` 호출
3. **역직렬화 로직**: 타입 태그를 읽고 해당 Generator의 `deserialize()` 호출

등록된 Generator 타입들:

- `DummyProofGenerator` : 재귀 증명용 더미 Generator
- `ArithmeticBaseGenerator` : 산술 게이트용 기본 Generator
- `ConstantGenerator` : 상수 값 Generator
- `PoseidonGenerator` : Poseidon 해시 Generator
- `PoseidonMdsGenerator` : Poseidon MDS 행렬 Generator
- `RandomValueGenerator` : 랜덤 값 Generator
- `SquareRootGenerator` : 우리가 정의한 Custom Generator

사용 방법:

```
let gate_serializer = DefaultGateSerializer;
let generator_serializer = CustomGeneratorSerializer::<C, D>::default();
```

```
// 직렬화
let data_bytes = data.to_bytes(&gate_serializer, &generator_serializer)?;


// 역직렬화
let data_from_bytes = CircuitData::<F, C, D>::from_bytes(
&data_bytes,
&gate_serializer,
&generator_serializer,
)?;
```

매크로 내부 동작 (개념적)

매크로가 생성하는 코드는 대략 다음과 같습니다:

```
// 타입 태그 정의
const TAG_DUMMY_PROOF: u8 = 0;
const TAG_ARITHMETIC_BASE: u8 = 1;
const TAG_CONSTANT: u8 = 2;
// ...
const TAG_SQUARE_ROOT: u8 = 6;


// 직렬화
fn write_generator(&self, dst: &mut Vec<u8>, generator: &dyn WitnessGenerator<F
if let Some(g) = generator.downcast_ref::<SquareRootGenerator<F, D>>() {
```

```
dst.write_u8(TAG_SQUARE_ROOT)?;

g.serialize(dst, common_data)?;

} else  if  let Some(g) = generator.downcast_ref::<ConstantGenerator<F>>() {

dst.write_u8(TAG_CONSTANT)?;

g.serialize(dst, common_data)?;

}

// ... 다른 타입들

}



// 역직렬화

fn  read_generator(&self, src: &mut Buffer) -> IoResult<Box<dyn WitnessGenerator

let  tag = src.read_u8()?;

match  tag {

TAG_SQUARE_ROOT => Ok(Box::new(SquareRootGenerator::deserialize(src, common_data

TAG_CONSTANT => Ok(Box::new(ConstantGenerator::deserialize(src, common_data)?)),

// ... 다른 타입들

_ => Err(IoError::new(ErrorKind::InvalidData, "Unknown generator type")),

}

}
```

main 함수 상세 분석

1. 타입 설정

```
const D: usize = 2;

type C = PoseidonGoldilocksConfig;

type F = <C as GenericConfig<D>>::F;
```

- `D = 2` : 2차 확장체 사용
- `PoseidonGoldilocksConfig` : Poseidon 해시 + Goldilocks Field
- `F` : Goldilocks Field 타입

2. 회로 구성

```
let  config = CircuitConfig::standard_recursion_config();

let  mut  builder = CircuitBuilder::<F, D>::new(config);


let  x = builder.add_virtual_target();

let  x_squared = builder.square(x);


builder.register_public_input(x_squared);
```

회로 구조:

```
x (비공개) ——> [Square Gate] ——> x_squared (공개)
```

`builder.square(x)` :

- `x * x` 를 계산하는 제약조건 추가
- 내부적으로 `ArithmeticGate` 사용

- 제약조건: `x * x = x_squared`

공개 입력:

- `x_squared` 만 공개 입력으로 등록
- `x` 는 비공개로 유지 (영지식 속성)
- 증명자는 "나는 x_squared의 제곱근을 알고 있다"를 증명

3. Custom Generator 추가

```
builder.add_simple_generator(SquareRootGenerator::<F, D> {
x,
x_squared,
_phantom: PhantomData,
});
```

`add_simple_generator()` :

- Generator를 회로에 등록
- Witness 생성 시 자동으로 실행됨
- Generator 인스턴스를 생성하여 전달

Generator 설정:

- `x` : 계산할 Target (제곱근)
- `x_squared` : 입력 Target (제곱 값)
- `_phantom` : 타입 마커

4. 이차잉여 생성

```
let x_squared_value = {
let mut val = F::rand();
while !val.is_quadratic_residue() {
val = F::rand();
}
val
};
```

이차잉여(Quadratic Residue)란?

필드 원소 `a` 가 이차잉여라는 것은 어떤 `x` 에 대해 `x^2 = a` 를 만족한다는 의미입니다.

왜 이차잉여를 선택해야 하는가?

- 모든 필드 원소가 제곱근을 가지는 것은 아님
- 이차잉여가 아닌 값을 선택하면 `sqrt()` 호출 시 `None` 반환
- `unwrap()` 에서 패닉 발생

Goldilocks Field에서의 이차잉여:

- 약 절반의 원소가 이차잉여
- `is_quadratic_residue()` 메서드로 확인 가능
- Legendre 기호를 사용하여 효율적으로 판별

코드 동작:

1. 랜덤 필드 원소 생성
2. 이차잉여인지 확인
3. 아니면 다시 랜덤 생성 (평균 2회 시도)

5. Witness 설정

```rust
let mut pw = PartialWitness::new();
pw.set_target(x_squared, x_squared_value)?;
```

중요한 점:

- `x_squared` 만 설정하고 `x` 는 설정하지 않음
- `x` 의 값은 `SquareRootGenerator` 가 자동으로 계산
- 이것이 Generator의 핵심 기능

Witness 생성 흐름:

```
1. 사용자: x_squared = (어떤 이차잉여) 설정
2. generate_partial_witness() 호출
3. SquareRootGenerator 실행:
- x_squared 값 읽기
- x = sqrt(x_squared) 계산
- x 값 설정
4. 회로 제약조건 검증: x * x = x_squared
```

6. 회로 빌드 및 증명 생성

```rust
let data = builder.build::<C>();
let proof = data.prove(pw.clone())?;
```

빌드 과정:

- 모든 게이트와 제약조건 정리
- Generator 리스트 저장
- FFT 준비 데이터 생성

증명 생성 과정:

1. `generate_partial_witness()` 호출

- 의존성 그래프 구성
- Generator들을 위상 정렬 순서로 실행
- `SquareRootGenerator.run_once()` 실행
- 모든 와이어 값 계산

2. Wire Commitment 생성
3. Permutation Argument
4. Quotient Polynomial 계산
5. FRI Proof 생성

7. 결과 출력 및 검증

```
let  x_squared_actual = proof.public_inputs[0];
println!("Field element (square): {x_squared_actual}");

data.verify(proof)
```

출력 예시:

```
Square root: 12345678901234567890

Field element (square): 15241578750190521121
```

검증 과정:

- 공개 입력 해시 검증
- 제약조건 검증 (x * x = x_squared)
- FRI 검증
- 모든 검증 통과 시 `Ok(())` 반환

8. Serialization 테스트

```
{
let gate_serializer = DefaultGateSerializer;
let generator_serializer = CustomGeneratorSerializer::<C, D>::default();

let data_bytes = data
.to_bytes(&gate_serializer, &generator_serializer)
.map_err(|_| anyhow::Error::msg("CircuitData serialization failed."))?;

let data_from_bytes = CircuitData::<F, C, D>::from_bytes(
&data_bytes,
&gate_serializer,
&generator_serializer,
)
```

```
.map_err(|_| anyhow::Error::msg("CircuitData deserialization failed."))?;


assert_eq!(data, data_from_bytes);

}
```

목적: 회로 데이터의 직렬화/역직렬화가 올바르게 동작하는지 확인합니다.

테스트 과정:

1. **Serializer 생성**:

```
let gate_serializer = DefaultGateSerializer;

let generator_serializer = CustomGeneratorSerializer::<C, D>::default();
```

- `DefaultGateSerializer` : 표준 게이트들을 직렬화
- `CustomGeneratorSerializer` : 우리가 정의한 Generator 포함

2. **직렬화**:

```
let data_bytes = data.to_bytes(&gate_serializer, &generator_serializer)?;
```

- `CircuitData` 를 바이트 배열로 변환
- 모든 게이트, Generator, 설정 등 포함

3. **역직렬화**:

```
let data_from_bytes = CircuitData::<F, C, D>::from_bytes(

&data_bytes,

&gate_serializer,
```

```
&generator_serializer,
)?;
```

- 바이트 배열에서 `CircuitData` 복원
- 동일한 Serializer 사용 필수

4. **검증**:

```
assert_eq!(data, data_from_bytes);
```

- 원본과 복원된 데이터가 동일한지 확인
- `CircuitData` 는 `PartialEq` trait 구현

중요성:

- 회로를 파일에 저장하고 나중에 로드 가능
- 네트워크를 통해 회로 전송 가능
- 검증자는 회로를 다시 빌드하지 않고 직렬화된 데이터만 로드

Generator의 역할 및 Witness 생성 과정

Witness 생성 파이프라인

```
1. PartialWitness 생성
↓
2. 사용자가 일부 Target 설정 (x_squared)
↓
3. generate_partial_witness() 호출
```

```
↓

4. Generator 의존성 그래프 구성

↓

5. 위상 정렬 (Topological Sort)

↓

6. Generator들을 순서대로 실행

├─> SquareRootGenerator.run_once()

├─> 다른 Generator들...

└─> 모든 Target 값 계산

↓

7. 전체 Witness 완성

↓

8. 제약조건 검증
```

의존성 그래프 예시

복잡한 회로에서는 여러 Generator가 서로 의존할 수 있습니다:

```
Target A (사용자 설정)

↓

Generator 1 → Target B

↓

Generator 2 → Target C

↓ ↓
```

```
└──────────┘
↓
Generator 3 → Target D
```

실행 순서: Generator 1 → Generator 2 → Generator 3

generate_partial_witness() 내부 동작

```rust
pub fn generate_partial_witness<F, C, D>(
inputs: PartialWitness<F>,
prover_data: &ProverOnlyCircuitData<F, C, D>,
common_data: &CommonCircuitData<F, D>,
) -> Result<PartitionWitness<F>> {
let mut witness = PartitionWitness::new(common_data.degree());
// 1. 사용자 입력 복사
for (target, value) in inputs.target_values {
witness.set_target(target, value)?;
}
// 2. Generator 의존성 그래프 구성
let mut dependency_graph = build_dependency_graph(&prover_data.generators);
// 3. 위상 정렬
let sorted_generators = topological_sort(&dependency_graph)?;
// 4. Generator 실행
for generator in sorted_generators {
```

```
let mut out_buffer = GeneratedValues::new();
generator.run_once(&witness, &mut out_buffer)?;
// 생성된 값들을 Witness에 추가
for (target, value) in out_buffer.target_values {
witness.set_target(target, value)?;
}
}
// 5. 모든 와이어 값이 설정되었는지 확인
ensure!(witness.is_complete(), "Witness generation incomplete");
Ok(witness)
}
```

순환 의존성 감지

만약 Generator A가 B에 의존하고, B가 A에 의존하면 순환 의존성이 발생합니다:

```
Generator A → Target X → Generator B → Target Y → Generator A (순환!)
```

이 경우 위상 정렬이 실패하고 에러가 발생합니다:

```
Error: Circular dependency detected in witness generators
```

실행 결과

실행 방법

```
cd plonky2/plonky2

cargo run --release --example square_root
```

예상 출력

```
Square root: 12345678901234567890

Field element (square): 15241578750190521121

Proof generated successfully

Verification successful
```

출력 설명:

- 첫 번째 줄: Generator가 계산한 제곱근 (비공개)
- 두 번째 줄: 공개 입력 (제곱 값)
- 증명 생성 및 검증 성공

성능 특성

회로 크기:

- Gate 수: 매우 적음 (주로 1개의 곱셈 게이트)
- 제약조건: `x * x = x_squared` (1개)
- Degree: 2 (곱셈)

증명 생성 시간:

- 이 간단한 예제: ~10-30ms (릴리스 빌드)
- Generator 실행 시간: ~1μs (제곱근 계산)

- 대부분의 시간은 FFT와 Merkle 트리 구성에 소요

증명 크기:

- 약 45-100 KB (회로 크기와 거의 무관)
- Plonky2의 간결한 증명 크기 특성

메모리 사용:

- 증명 생성: ~50-100 MB
- 검증: ~10-20 MB

다른 예제와의 비교

Fibonacci 예제와의 차이점

| 특징 | Fibonacci | Square Root |

|------|-----------|-------------|

| 주요 연산 | 덧셈 (회로 내) | 제곱근 (회로 외) |

| Generator 사용 | 없음 (자동 생성) | Custom Generator |

| Witness 설정 | 초기값만 설정 | 제곱 값만 설정 |

| 회로 복잡도 | 높음 (99개 덧셈) | 낮음 (1개 곱셈) |

| 계산 위치 | 회로 내 | 회로 외 (Generator) |

Range Check 예제와의 차이점

| 특징 | Range Check | Square Root |

|------|-------------|-------------|

| Generator | 내장 (WireSplitGenerator) | Custom |

| 목적 | 범위 증명 | 제곱근 지식 증명 |

| 제약조건 | 비트 분해 + 합 검증 | 곱셈 검증 |

| 복잡도 | 중간 (비트 수에 비례) | 낮음 (상수) |

Custom Generator 설계 가이드라인

언제 Custom Generator를 사용해야 하는가?

사용해야 하는 경우:

1. **복잡한 계산**:

- 제곱근, 나눗셈, 모듈러 역원
- 회로로 표현하면 너무 많은 게이트 필요
- 예: `x = a / b` (나눗셈은 비싸지만, `x * b = a` 검증은 저렴)

2. **비결정적 계산**:

- 여러 가능한 답 중 하나를 선택
- 예: 이차잉여의 두 제곱근 중 하나 선택

3. **외부 데이터 접근**:

- 파일, 데이터베이스, 네트워크에서 데이터 읽기
- 예: Merkle 증명 생성 시 트리 데이터 접근

4. **최적화**:

- 회로 크기를 줄이기 위해 일부 계산을 외부로 이동
- 예: 큰 정수 연산을 필드 연산으로 분해

사용하지 말아야 하는 경우:

1. **간단한 산술 연산**: 덧셈, 곱셈 등은 회로에서 직접 처리

2. **보안이 중요한 계산**: Generator는 검증되지 않으므로 신뢰 필요

3. **결정적 계산**: 회로 제약조건만으로 충분한 경우

Custom Generator 설계 패턴

패턴 1: 역연산 (Inverse Operation)

회로에서 검증하기 쉬운 연산의 역연산을 Generator로 수행:

```
// 회로: x * y = z (검증)
// Generator: y = z / x (계산)

struct DivisionGenerator {
x: Target, // 입력
z: Target, // 입력
y: Target, // 출력 (계산할 값)
}

impl SimpleGenerator for DivisionGenerator {
fn dependencies(&self) -> Vec<Target> {
vec![self.x, self.z]
}
fn run_once(&self, witness: &PartitionWitness, out_buffer: &mut GeneratedValues
let x = witness.get_target(self.x);
let z = witness.get_target(self.z);
let y = z * x.inverse()?; // y = z / x
out_buffer.set_target(self.y, y)
```

```
}
}
```

패턴 2: 분해 (Decomposition)

복잡한 값을 여러 부분으로 분해:

```
// 회로: x = low + high * 2^32 (검증)
// Generator: (low, high) = decompose(x) (계산)

struct DecomposeGenerator {
x: Target, // 입력
low: Target, // 출력
high: Target, // 출력
}

impl SimpleGenerator for DecomposeGenerator {
fn dependencies(&self) -> Vec<Target> {
vec![self.x]
}
fn run_once(&self, witness: &PartitionWitness, out_buffer: &mut GeneratedValues
let x = witness.get_target(self.x).to_canonical_u64();
let low = x & 0xFFFFFFFF;
let high = x >> 32;
```

```
out_buffer.set_target(self.low, F::from_canonical_u64(low))?;
out_buffer.set_target(self.high, F::from_canonical_u64(high))
}
}
```

패턴 3: 외부 데이터 (External Data)

외부 소스에서 데이터를 가져와 Witness에 제공:

```
// 회로: hash(preimage) = digest (검증)
// Generator: preimage = lookup(digest) (계산)

struct PreimageGenerator {
digest: Target,
preimage: Vec<Target>,
database: Arc<HashMap<Hash, Vec<u8>>>, // 외부 데이터
}

impl SimpleGenerator for PreimageGenerator {
fn  dependencies(&self) -> Vec<Target> {
vec![self.digest]
}
fn  run_once(&self, witness: &PartitionWitness, out_buffer: &mut GeneratedValues
let  digest = witness.get_target(self.digest);
```

```
let preimage_bytes = self.database.get(&digest)
.ok_or_else(|| anyhow!("Preimage not found"))?;
for (target, &byte) in self.preimage.iter().zip(preimage_bytes) {
out_buffer.set_target(*target, F::from_canonical_u8(byte))?;
}
Ok(())
}
}
```

Generator 구현 시 주의사항

1. 의존성 정확히 명시

```
// ❌ 잘못된 예: 의존성 누락
fn dependencies(&self) -> Vec<Target> {
vec![] // x_squared를 읽는데 의존성에 포함 안 함
}

// ✅ 올바른 예
fn dependencies(&self) -> Vec<Target> {
vec![self.x_squared] // 읽는 모든 Target 포함
}
```

2. 에러 처리

```
// ❌ 잘못된 예: 패닉 발생
fn run_once(&self, witness: &PartitionWitness, out_buffer: &mut GeneratedValues
let x = witness.get_target(self.x_squared).sqrt().unwrap(); // 패닉 가능
out_buffer.set_target(self.x, x)
}


// ✅ 올바른 예: 에러 반환
fn run_once(&self, witness: &PartitionWitness, out_buffer: &mut GeneratedValues
let x_squared = witness.get_target(self.x_squared);
let x = x_squared.sqrt()
.ok_or_else(|| anyhow!("Square root does not exist"))?;
out_buffer.set_target(self.x, x)
}
```

3. Serialization 대칭성

```
// ✅ 올바른 예: serialize와 deserialize가 대칭
fn serialize(&self, dst: &mut Vec<u8>, _: &CommonCircuitData<F, D>) -> IoResult
dst.write_target(self.x)?;
dst.write_target(self.x_squared)?;
dst.write_u64(self.some_config) // 추가 필드
}
```

```
fn deserialize(src: &mut Buffer, _: &CommonCircuitData<F, D>) -> IoResult<Self>
let x = src.read_target()?;
let x_squared = src.read_target()?;
let some_config = src.read_u64()?; // 동일한 순서로 읽기
Ok(Self { x, x_squared, some_config, _phantom: PhantomData })
}
```

4. 결정론적 동작

```
// ❌ 잘못된 예: 비결정적 (랜덤 사용)
fn run_once(&self, witness: &PartitionWitness, out_buffer: &mut GeneratedValues
let random_value = F::rand(); // 매번 다른 값
out_buffer.set_target(self.output, random_value)
}

// ✅ 올바른 예: 결정론적 (입력에만 의존)
fn run_once(&self, witness: &PartitionWitness, out_buffer: &mut GeneratedValues
let input = witness.get_target(self.input);
let output = compute_deterministic(input); // 항상 같은 결과
out_buffer.set_target(self.output, output)
}
```

예외: `RandomValueGenerator` 처럼 의도적으로 랜덤 값을 생성하는 경우는 제외

5. 성능 고려

```
// ❌ 비효율적: 불필요한 계산
fn run_once(&self, witness: &PartitionWitness, out_buffer: &mut GeneratedValues
for i in 0..1000000 {
// 복잡한 계산...
}
// ...
}

// ✅ 효율적: 필요한 계산만
fn run_once(&self, witness: &PartitionWitness, out_buffer: &mut GeneratedValues
// 최소한의 계산으로 결과 도출
// ...
}
```

Generator는 증명 생성 시마다 실행되므로 성능이 중요합니다.

고급 주제

1. 복잡한 Generator 체인

여러 Generator가 연쇄적으로 실행되는 경우:

```
// Generator 1: a를 계산
```

```rust
struct GeneratorA {
input: Target,
a: Target,
}

// Generator 2: b를 계산 (a에 의존)
struct GeneratorB {
a: Target,
b: Target,
}

// Generator 3: c를 계산 (a와 b에 의존)
struct GeneratorC {
a: Target,
b: Target,
c: Target,
}

impl SimpleGenerator for GeneratorC {
fn  dependencies(&self) -> Vec<Target> {
vec![self.a, self.b] // 두 개의 의존성
}
// ...
```

```
}
```

실행 순서: Generator A → Generator B → Generator C

2. 조건부 Generator

특정 조건에서만 실행되는 Generator:

```
struct ConditionalGenerator {
condition: Target,
input: Target,
output: Target,
}

impl SimpleGenerator for ConditionalGenerator {
fn  run_once(&self, witness: &PartitionWitness, out_buffer: &mut GeneratedValues
let  condition = witness.get_target(self.condition);
if  condition == F::ONE {
let  input = witness.get_target(self.input);
let  output = compute(input);
out_buffer.set_target(self.output, output)?;
} else {
// 조건이 거짓이면 기본값 설정
out_buffer.set_target(self.output, F::ZERO)?;
}
```

```
Ok(())
}
}
```

3. 배치 Generator

여러 값을 한 번에 계산하는 Generator:

```
struct BatchSquareRootGenerator {
inputs: Vec<Target>,
outputs: Vec<Target>,
}

impl SimpleGenerator for BatchSquareRootGenerator {
fn  dependencies(&self) -> Vec<Target> {
self.inputs.clone()
}
fn  run_once(&self, witness: &PartitionWitness, out_buffer: &mut GeneratedValues
for (input, output) in  self.inputs.iter().zip(&self.outputs) {
let  x_squared = witness.get_target(*input);
let  x = x_squared.sqrt()
.ok_or_else(|| anyhow!("Square root does not exist"))?;
out_buffer.set_target(*output, x)?;
}
```

```
Ok(())
}
}
```

4. 상태를 가진 Generator

외부 상태를 참조하는 Generator (주의해서 사용):

```
struct StatefulGenerator {
output: Target,
state: Arc<Mutex<SomeState>>, // 공유 상태
}

impl SimpleGenerator for StatefulGenerator {
fn  run_once(&self, witness: &PartitionWitness, out_buffer: &mut GeneratedValues
let  state = self.state.lock().unwrap();
let  value = state.compute_value();
out_buffer.set_target(self.output, value)
}
}
```

주의: 상태를 가진 Generator는 직렬화가 복잡하고 재현성 문제가 있을 수 있습니다.

실전 응용 사례

1. 모듈러 역원 (Modular Inverse)

```
struct ModularInverseGenerator {
x: Target,
x_inv: Target,
}

impl SimpleGenerator for ModularInverseGenerator {
fn  dependencies(&self) -> Vec<Target> {
vec![self.x]
}
fn  run_once(&self, witness: &PartitionWitness, out_buffer: &mut GeneratedValues
let  x = witness.get_target(self.x);
let  x_inv = x.try_inverse()
.ok_or_else(|| anyhow!("Inverse does not exist"))?;
out_buffer.set_target(self.x_inv, x_inv)
}
}

// 회로에서 검증: x * x_inv = 1
let  one = builder.one();
let  product = builder.mul(x, x_inv);
```

```
builder.connect(product, one);
```

2. 큰 정수 나눗셈

```
struct BigIntDivisionGenerator {
dividend_limbs: Vec<Target>,
divisor_limbs: Vec<Target>,
quotient_limbs: Vec<Target>,
remainder_limbs: Vec<Target>,
}

impl SimpleGenerator for BigIntDivisionGenerator {
fn  run_once(&self, witness: &PartitionWitness, out_buffer: &mut GeneratedValues
// 큰 정수를 limb들로부터 재구성
let  dividend = reconstruct_bigint(witness, &self.dividend_limbs);
let  divisor = reconstruct_bigint(witness, &self.divisor_limbs);
// 나눗셈 수행
let (quotient, remainder) = dividend.div_rem(&divisor);
// 결과를 limb들로 분해하여 저장
set_bigint_limbs(out_buffer, &self.quotient_limbs, &quotient)?;
set_bigint_limbs(out_buffer, &self.remainder_limbs, &remainder)?;
Ok(())
}
```

```
}

// 회로에서 검증: dividend = quotient * divisor + remainder
```

3. Merkle 증명 생성

```
struct MerkleProofGenerator {
leaf: Target,
leaf_index: Target,
proof_siblings: Vec<Target>,
merkle_tree: Arc<MerkleTree>, // 외부 데이터
}

impl SimpleGenerator for MerkleProofGenerator {
fn dependencies(&self) -> Vec<Target> {
vec![self.leaf, self.leaf_index]
}
fn run_once(&self, witness: &PartitionWitness, out_buffer: &mut GeneratedValues
let leaf = witness.get_target(self.leaf);
let index = witness.get_target(self.leaf_index).to_canonical_u64() as usize;
// Merkle 트리에서 증명 경로 가져오기
let proof = self.merkle_tree.get_proof(index)?;
```

```
// 증명의 각 sibling을 Target에 설정
for (target, &sibling) in  self.proof_siblings.iter().zip(&proof.siblings) {
out_buffer.set_target(*target, sibling)?;
}
Ok(())
}
}

// 회로에서 검증: Merkle 경로가 올바른지 확인
```

4. ECDSA 서명 검증 보조

```
struct ECDSAPointGenerator {
scalar: Target,
point_x: Target,
point_y: Target,
}

impl SimpleGenerator for ECDSAPointGenerator {
fn  dependencies(&self) -> Vec<Target> {
vec![self.scalar]
}
fn  run_once(&self, witness: &PartitionWitness, out_buffer: &mut GeneratedValues
```

```
let  scalar = witness.get_target(self.scalar);

// 타원곡선 점 곱셈 수행 (회로 외부에서)

let  point = scalar_mult(&G, scalar);

out_buffer.set_target(self.point_x, point.x)?;

out_buffer.set_target(self.point_y, point.y)?;

Ok(())

}

}



// 회로에서 검증: 점이 곡선 위에 있고 올바른 관계를 만족하는지 확인
```

학습 포인트

이 예제를 통해 다음을 학습할 수 있습니다:

1. Generator의 핵심 개념

- Generator는 회로 외부에서 복잡한 계산을 수행
- 회로는 계산 결과를 검증만 함
- 이를 통해 회로 크기를 크게 줄일 수 있음

2. SimpleGenerator Trait

- `id()` : Generator 식별
- `dependencies()` : 실행 순서 결정

- `run_once()` : 실제 계산 수행
- `serialize()` / `deserialize()` : 영속성 지원

3. Witness 생성 과정

- 사용자가 일부 값만 설정
- Generator들이 나머지 값을 자동 계산
- 의존성 그래프를 통한 순서 결정
- 위상 정렬로 실행 순서 보장

4. 회로 설계 패턴

- 복잡한 계산은 Generator로 이동
- 회로는 간단한 검증만 수행
- 예: 제곱근 계산 (복잡) vs 곱셈 검증 (간단)

5. Serialization의 중요성

- 회로를 저장하고 로드할 수 있음
- Generator도 함께 직렬화되어야 함
- CustomGeneratorSerializer로 여러 Generator 타입 지원

6. 실전 응용

- 나눗셈, 역원, 제곱근 등 역연산
- 큰 정수 연산
- 외부 데이터 접근 (Merkle 트리 등)
- 암호학적 연산 보조 (ECDSA 등)

다음 단계

더 고급 주제를 학습하려면 다음 문서를 참조하세요:

- 10-recursive-proofs.md: 재귀 증명 구성법
- 11-performance.md: 성능 최적화 기법
- 02-circuit-building.md: CircuitBuilder 고급 기능

참고 자료

- 07-example-fibonacci.md: Fibonacci 예제 (Generator 없음)
- 08-example-range-check.md: Range Check (내장 Generator)
- 03-proof-generation.md: 증명 생성 프로세스
- Plonky2 GitHub: https://github.com/0xPolygonZero/plonky2

요약

Square Root 예제는 Custom Generator의 강력함을 보여줍니다:

1. **효율성**: 제곱근 계산을 회로 외부로 이동하여 회로 크기 최소화
2. **유연성**: 복잡한 계산을 Rust 코드로 자유롭게 구현
3. **검증 가능성**: 회로는 결과만 검증하여 정확성 보장
4. **재사용성**: Generator를 다른 회로에서도 재사용 가능

이 패턴은 실전 영지식증명 애플리케이션에서 매우 중요하며, 복잡한 계산을 효율적으로 처리하는 핵심 기법입니다.

문서 네비게이션

- **이전**: 08. Range Check 예제 - Lookup 사용 예제
- **다음**: 10. 재귀 증명 - 재귀 증명 구성
- **관련 문서**:
- 02. 회로 구성 프로세스 - CircuitBuilder 기본
- 07. Fibonacci 예제 - 기본 예제
- 08. Range Check 예제 - 중급 예제
- 03. 증명 생성 프로세스 - Witness 생성 과정